Python for Artificial Intelligence

Build the Support Desk Chatbot

Week 5, Day 5

PROJECT

Today's Project: The Support Desk Bot

- Question: can a program work out what a customer wants from a one-line message, and answer it?
- The data is the Bitext customer-support dataset: 26,872 short messages, each labelled with one of 27 intents such as get_refund, track_order or recover_password
- Hour 1: turn text into numbers with n-grams, bag of words and TF-IDF. Hour 2: train a classifier and check whether its score can be trusted
- Hour 3: add confidence, prepared replies and a chat loop. By the end you have a working chatbot with no LLM in it, and a plan for making it better

RECAP

What We Bring From Days 23 and 24

The chatbot uses almost everything we have learned so far.

Tool	Where it appears in the bot
Regex cleaning	clean(): lowercase, replace {{slots}} with the word slot, strip symbols
Tokens and vocabulary	TF-IDF builds the vocabulary of words the classifier sees
Stop words, stemming, lemmas	Optional: we test them against the plain version instead of assuming they help
Regex and entities	Practice: pull an order number out of a message and use it in the reply
Bag of words and TF-IDF	Turn each message into a row of numbers
Classifier from Week 4	Logistic Regression, now with 27 classes

Hour 1

Data and Numbers

EXAMPLE

Meet the Dataset

On Kaggle, click Add Input and search for "Bitext customer support". The first loop prints the path of every file you have added, so you can copy the CSV's path.

```
import os, re, random
import numpy as np, pandas as pd

for folder, _, files in os.walk('/kaggle/input'):         # find the CSV on Kaggle
    for f in files: print(os.path.join(folder, f))

df = pd.read_csv('/kaggle/input/bitext-gen-ai-chatbot-customer-support-dataset/'
                 'Bitext_Sample_Customer_Support_Training_Dataset_27K_responses-v11.csv')
print(df.shape)                           # (26872, 5)
print(df.columns.tolist())                # ['flags', 'instruction', 'category', 'intent',
'response']
print(df['intent'].nunique(), df['category'].nunique())     # 27 11
```

EXAMPLE

A Look at the Data

Always look at real rows first. What does a message look like, and how many of each intent are there?

```
print(df[['instruction', 'intent']].sample(5, random_state=1).to_string())

counts = df['intent'].value_counts()
print(counts.min(), counts.max())                                  # 950 1000   about 1,000 per
intent
print(df['instruction'].str.split().str.len().mean().round(1))  # 8.7  words per message
print(df['instruction'].str.contains('{{', regex=False).mean().round(2))   # 0.25   with a slot
```

CONCEPT

What This Data Is, and Is Not

- The messages were generated by a language-technology company from real-life text, then expanded with variations: typos, slang, short and long forms. Real customers will be messier
- About 1,000 messages per intent, so the classes are balanced. That is rare in real data and makes this a friendly dataset
- A quarter of the messages contain a slot such as {{Order Number}}: a placeholder where a real order number would go
- Every row also has a ready-made reply in the response column. We will reuse those for the bot in Hour 3

EXAMPLE

Cleaning Text

Real messages are messy. We lowercase, replace every {{slot}} with one shared word, and strip punctuation and digits.

```
def clean(text):
    text = text.lower()
    text = re.sub(r'\{\{.*?\}\}', ' slot ', text)      # {{Order Number}} -> slot
    text = re.sub(r'[^a-z ]', ' ', text)               # keep only letters and spaces
    return ' '.join(text.split())                       # tidy up the spaces

print(clean("I'm rtying to cancel order {{Order Number}}"))    # i m rtying to cancel order slot
print(clean('how can I check bill  #12588?'))                  # how can i check bill

df['clean'] = df['instruction'].apply(clean)
```

WHY, WHAT, LIMITS

Before We Start: N-grams

Why it matters, what it is, and where it stops working.

	N-grams
Why it matters	Single words lose order. 'Not working' and 'working' mean opposite things. N-grams keep short runs of words together so a little order survives
What it is	An n-gram is a run of n consecutive words: a unigram is one word, a bigram two, a trigram three
Limitations	The number of features grows very fast (1,199 to 15,361 on our data), most n-grams are rare, and they still only see a short window of context

CONCEPT

N-grams: Keeping a Little Word Order

- An n-gram is a run of n consecutive words. 1 word is a unigram, 2 a bigram, 3 a trigram
- "where is my refund" has the bigrams (where, is), (is, my), (my, refund)
- Bag of words loses order. Bigrams bring a little back: "not working" becomes one feature, not two separate words
- The cost: the number of features grows fast, and most n-grams are rare

EXAMPLE

Generating N-grams

A sliding window of size n over the tokens.

```
def ngrams(text, n):
    tokens = text.split()
    return [tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]

text = "where is my refund"
print(ngrams(text, 1))
print(ngrams(text, 2))
print(ngrams(text, 3))
# >> [('where',), ('is',), ('my',), ('refund',)]
#    [('where', 'is'), ('is', 'my'), ('my', 'refund')]
#    [('where', 'is', 'my'), ('is', 'my', 'refund')]
```

EXAMPLE

N-grams Make the Vocabulary Explode

CountVectorizer builds the n-gram vocabulary for us. min_df=2 drops anything seen in only one message. Compare the feature counts on our 26,872 messages.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

df = pd.read_csv('bitext.csv')
msgs = df['instruction'].str.lower()

for rng in [(1, 1), (1, 2), (1, 3)]:
    v = CountVectorizer(ngram_range=rng, min_df=2)
    v.fit(msgs)
    print(rng, len(v.vocabulary_), 'features')
# >> (1, 1) 1199 features
#    (1, 2) 5767 features
#    (1, 3) 15361 features
```

CONCEPT

Vectorization: Words to Numbers

- A model needs numbers, so every text becomes a vector. There are three main ways
- Bag of Words: count how often each vocabulary word appears. Order is ignored
- TF-IDF: like counting, but weigh each word by how rare it is across all texts, so rare, informative words count more
- Word embeddings (Word2Vec, GloVe, and the ones inside LLMs): each word is a short dense vector, where similar words get similar vectors. We meet these in the bridge session

WHY, WHAT, LIMITS

Before We Start: Bag of Words

Why it matters, what it is, and where it stops working.

	Bag of Words
Why it matters	Models need numbers. Counting how many times each vocabulary word appears is the simplest way to turn a message into a row of numbers
What it is	Bag of words builds a vocabulary of all distinct words and represents each text as a row of counts, one per word
Limitations	Word order is lost, every word counts equally, the table is huge and mostly zeros, and 'refund' and 'reimbursement' look unrelated

CONCEPT

Bag of Words: Its Two Parts

- Vocabulary: the list of all unique words in the whole dataset. Each word becomes one column (a feature)
- Document representation: each text becomes a row of counts, one count per vocabulary word
- Most counts are zero, so scikit-learn stores the table as a sparse matrix
- Weakness: word order is lost, and every word counts the same, so "my" counts as much as "refund"

EXAMPLE

Bag of Words

CountVectorizer builds a vocabulary and counts how many times each word appears in each message. Word order is thrown away, hence the name.

```
from sklearn.feature_extraction.text import CountVectorizer

demo = ['cancel my order', 'where is my order', 'i want a refund']
cv = CountVectorizer()
counts = cv.fit_transform(demo)          # a sparse table: one row per message
print(cv.get_feature_names_out())
# ['cancel' 'is' 'my' 'order' 'refund' 'want' 'where']   (the single letters a and i are skipped)
print(counts.toarray())
# [[1 0 1 1 0 0 0]
#  [0 1 1 1 0 0 1]
#  [0 0 0 0 1 1 0]]
```

CONCEPT

What Bag of Words Gives Us

- Each message becomes a row of numbers with one column per vocabulary word: exactly the table shape that Logistic Regression expects from Week 4
- Most entries are zero. A message has about 9 words but the vocabulary has thousands, so scikit-learn stores the table in a compact sparse format
- The cost: word order is gone. "dog bites man" and "man bites dog" give identical rows
- Another weakness: every word counts equally, so "my" matters as much as "refund". TF-IDF fixes that

WHY, WHAT, LIMITS

Before We Start: TF-IDF

Why it matters, what it is, and where it stops working.

	TF-IDF
Why it matters	In bag of words, 'my' counts as much as 'refund'. TF-IDF gives rare, informative words more weight, which usually makes classifiers more accurate
What it is	TF-IDF multiplies how often a word appears in this text by how rare it is across all texts
Limitations	It still ignores word order and meaning, so synonyms stay unrelated. Its weights depend on the dataset it was fitted on, so it must be fitted on training data only

CONCEPT

TF-IDF: The Idea

- Term Frequency (TF): how often a word appears in this text
- Inverse Document Frequency (IDF): lowers the weight of words that appear in many texts and raises the weight of rare ones
- TF-IDF = TF x IDF. A word that is frequent here but rare everywhere else scores highest
- Example: "refund" is a strong clue to an intent, "my" is not, because almost every message has "my"

EXAMPLE

TF-IDF on Three Tiny Messages

TfidfVectorizer learns the vocabulary and the idf values, then turns each message into a row of weights. Each row is scaled to length 1.

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["where is my refund", "where is my order", "refund refund refund"]
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(docs)
for w, idf in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(f"{w:7s} idf = {idf:.2f}")
print(X.toarray().round(2))
# >> is      idf = 1.29
#    my      idf = 1.29
#    order   idf = 1.69
#    refund  idf = 1.29
#    where   idf = 1.29
#    [[0.5  0.5  0.   0.5  0.5 ]
#     [0.46 0.46 0.6  0.   0.46]
#     [0.   0.   0.   1.   0.  ]]
```

CONCEPT

Reading the TF-IDF Output

- "order" has the highest idf (1.69): it appears in only one of the three messages. The other words each appear in two of the three, so they score lower
- In message 3, "refund" is the only word, so its weight is 1.0
- In message 2, "order" gets the largest weight (0.60), because it is the one word that tells message 2 apart
- Each row can now go into any model from Weeks 3 and 4

CONCEPT

What TF-IDF Is Used For

- Document similarity and clustering: group news, papers or support tickets by topic
- Text classification: spam filtering, sentiment analysis, intent detection (tomorrow)
- Keyword extraction: the highest-weighted words of a document are its keywords
- Search: rank documents by how well their TF-IDF vector matches the query

CONCEPT

TF-IDF: Weigh Words by How Rare They Are

TF-IDF multiplies how often a word appears in this message by how rare it is across all messages. The rarer the word, the more it tells us. These are real values from our 26,872 messages.

Word	Share of messages that contain it	IDF (rarity score)	What it means
refund	2%	4.91	Rare: a strong clue about the intent
cancel	3%	4.63	Rare: a strong clue
order	7%	3.63	Fairly common: a weaker clue
my	18%	2.73	Very common: says little
slot	25%	2.39	In a quarter of all messages: the weakest clue

EXAMPLE

TF-IDF on a Real Message

TfidfVectorizer does the weighting for us. Here are the five biggest weights for one message.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer().fit(df['clean'])
words = tfidf.get_feature_names_out()
row = tfidf.transform([clean('where is my refund for order {{Order Number}}')]).toarray()[0]

for i in row.argsort()[::-1][:5]:       # the 5 biggest weights in this message
    print(f'{words[i]:8s} {row[i]:.2f}')
# refund 0.49,  is 0.43,  where 0.42,  order 0.37,  for 0.36
```

QUICK CHECK

Quick Check

In the weights above, why does "order" get a lower weight than "refund", even though both appear once in the message?

ANSWER

Quick Check: Answer

```
# TF is the same (once each), so the difference comes from IDF.
# 'order' appears in 7% of all messages, so seeing it narrows
# things down only a little. 'refund' appears in just 2%, so it
# is a much stronger clue about what the customer wants.
#
# TF-IDF turns that idea into a number.
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 2

Classifying Messages and Building the Bot

EXAMPLE

Split and Vectorize

Same discipline as Week 4: learn the vocabulary from the training messages only, then apply it to the test messages. stratify keeps all 27 intents in the same proportions in both sets.

```
from sklearn.model_selection import train_test_split

X_train_text, X_test_text, y_train, y_test = train_test_split(
    df['clean'], df['intent'], test_size=0.2, stratify=df['intent'], random_state=42)

vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train_text)       # learn the vocabulary from train only
X_test = vectorizer.transform(X_test_text)             # reuse it on the test messages
print(X_train.shape, X_test.shape)    # (21497, 2510) (5375, 2510)
```

EXAMPLE

Train and Score the Classifier

Logistic Regression from Day 18, now with 27 classes instead of 2. scikit-learn handles the extra classes for us.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print((predictions == y_test).mean())                    # 0.9939   accuracy on the test messages
print(model.classes_[:3])                                # ['cancel_order' 'change_order'
'change_shipping_address']
```

PROJECT

99.4% Accuracy. Is That Believable?

- The model gets 99.4% of the 5,375 test messages right: 33 mistakes. Far higher than most real projects reach
- Be suspicious of a score this high, as on Day 22. This data was generated from templates, so messages of one intent look alike and the intents are easy to tell apart
- Real customer messages are messier, so expect a lower score in real life
- Next slide: check whether the test messages are just copies of training messages

EXAMPLE

Check for Copies

How many test messages also appear, word for word, in the training set? And how does the model do on the ones it has never seen?

```
seen = set(X_train_text)                                  # every cleaned training message
is_copy = np.array([m in seen for m in X_test_text])
print(is_copy.sum(), 'of', len(is_copy), 'test messages are in the training set')
# 889 of 5375 test messages are in the training set

fresh = ~is_copy                                          # messages the model never saw
print((predictions[fresh] == y_test[fresh]).mean())     # 0.9929   accuracy on those only
```

PROJECT

Reading the Copy Check

- 889 of 5,375 test messages (17%) are exact copies of a training message. That is leakage: the model has seen the answer
- On the remaining messages it has never seen, accuracy is still 99.3%. So copies do not explain the high score: the dataset itself is easy
- Both checks together are the honest picture: very good on this dataset, and unknown on real customers until we test with their messages
- Habit from Day 22: ask where a score comes from before you trust it

EXAMPLE

Which Intents Are Hardest?

classification_report gives precision, recall and F1 for every intent (Day 18). We sort by F1 and keep the five lowest.

```
from sklearn.metrics import classification_report

report = classification_report(y_test, predictions, output_dict=True)
scores = pd.DataFrame(report).T.iloc[:27]              # one row per intent
print(scores.sort_values('f1-score').head(5).round(3)) # the five hardest intents (next slide)
```

RESULTS

The Five Hardest Intents

Real output on the 5,375 test messages. Even the hardest intents score above 0.97, so the differences are tiny.

Intent	Precision	Recall	F1	Test messages
check_invoice	0.980	0.975	0.977	200
create_account	0.990	0.980	0.985	199
get_invoice	0.995	0.980	0.987	200
registration_problems	0.990	0.985	0.987	200
change_order	0.980	0.995	0.988	199

EXAMPLE

Reading the Mistakes

Set the diagonal of the confusion matrix to zero so only mistakes remain, then list the five biggest.

```
from sklearn.metrics import confusion_matrix

labels = model.classes_
cm = confusion_matrix(y_test, predictions, labels=labels)
np.fill_diagonal(cm, 0)                                  # keep only the mistakes
for i, j in zip(*np.unravel_index(cm.ravel().argsort()[::-1][:5], cm.shape)):
    print(f'{cm[i, j]:3d}  {labels[i]} -> {labels[j]}')
#   4  check_invoice -> delivery_options
#   4  get_invoice -> check_invoice
#   3  track_order -> change_order
#   3  create_account -> delete_account
#   2  payment_issue -> registration_problems
```

PROJECT

What the Mistakes Have in Common

- The top confusions are check_invoice to delivery_options (4), get_invoice to check_invoice (4), track_order to change_order (3) and create_account to delete_account (3)
- They are neighbours in meaning: checking an invoice and getting an invoice use almost the same words. A human might hesitate too
- Bag of words only counts words, so a short or unusual message that lacks the key words gives the model little to separate similar intents
- Always read a confusion matrix: it tells you which intents to improve or merge

EXAMPLE

Trying New Messages

predict_proba gives one probability per intent. The highest is the prediction, and it doubles as the model's confidence.

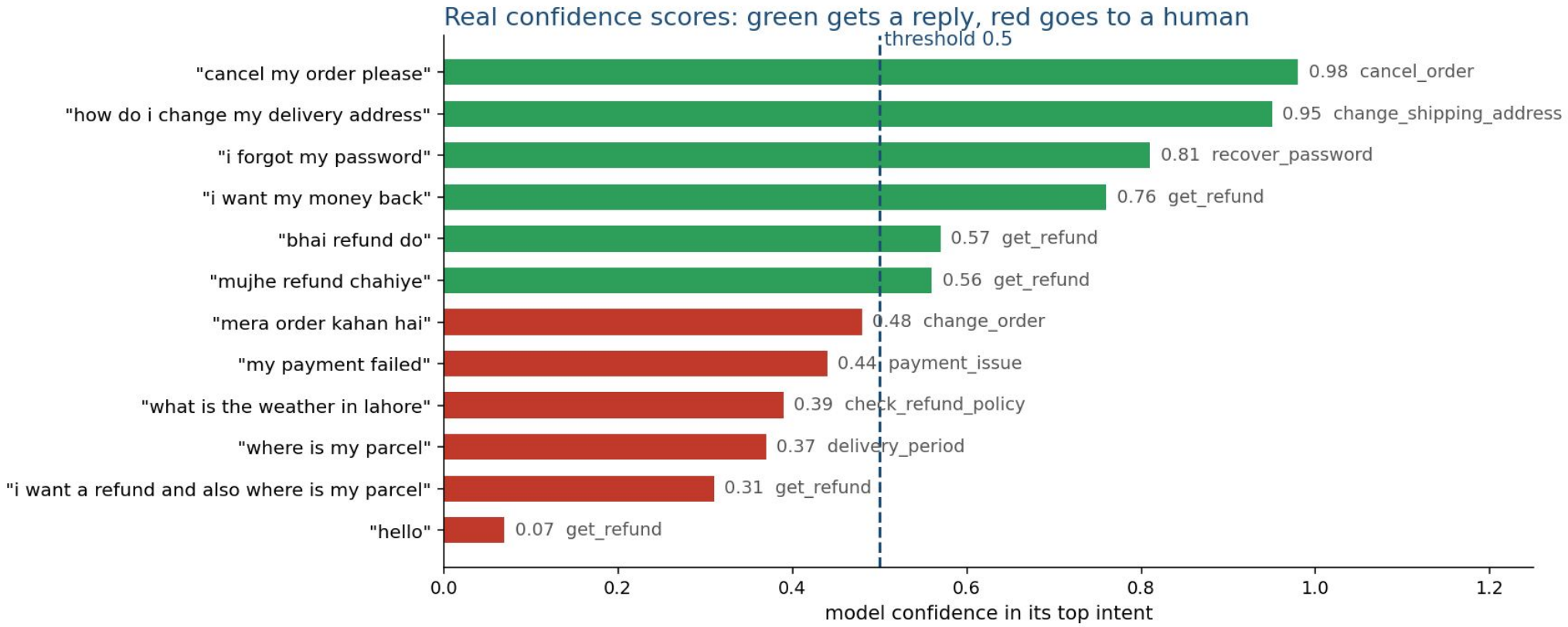
```
def top_intents(message, k=2):
    probs = model.predict_proba(vectorizer.transform([clean(message)]))[0]
    best = probs.argsort()[::-1][:k]
    return [(model.classes_[i], round(float(probs[i]), 2)) for i in best]

for text in ['i want my money back', 'where is my parcel', 'hello', 'what is the weather in
lahore']:
    print(text, '->', top_intents(text))
# i want my money back -> [('get_refund', 0.76), ('check_refund_policy', 0.14)]
# where is my parcel -> [('delivery_period', 0.37), ('recover_password', 0.11)]
# hello -> [('get_refund', 0.07), ('place_order', 0.06)]
# what is the weather in lahore -> [('check_refund_policy', 0.39), ('check_cancellation_fee',
0.09)]
```

RESULTS

Confidence Scores on Real Messages

The model always picks something, but its confidence tells us when to trust it. Messages that look like the training data score high. Casual, off-topic or two-in-one messages score low.



CONCEPT

Confidence and "I Don't Know"

- The model can only choose among its 27 intents. Asked about the weather it still picks one (check_refund_policy, 0.39), because it has no "none of these" option
- "hello" gets only 0.07, and "i want a refund and also where is my parcel" gets 0.31 because it contains two requests
- Roman Urdu: "mujhe refund chahiye" gets 0.56 for get_refund only because the English word "refund" appears. "mera order kahan hai" gets 0.48. The model never saw Roman Urdu
- So we add a rule: if the top confidence is below a threshold, do not answer. Hand over to a human

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

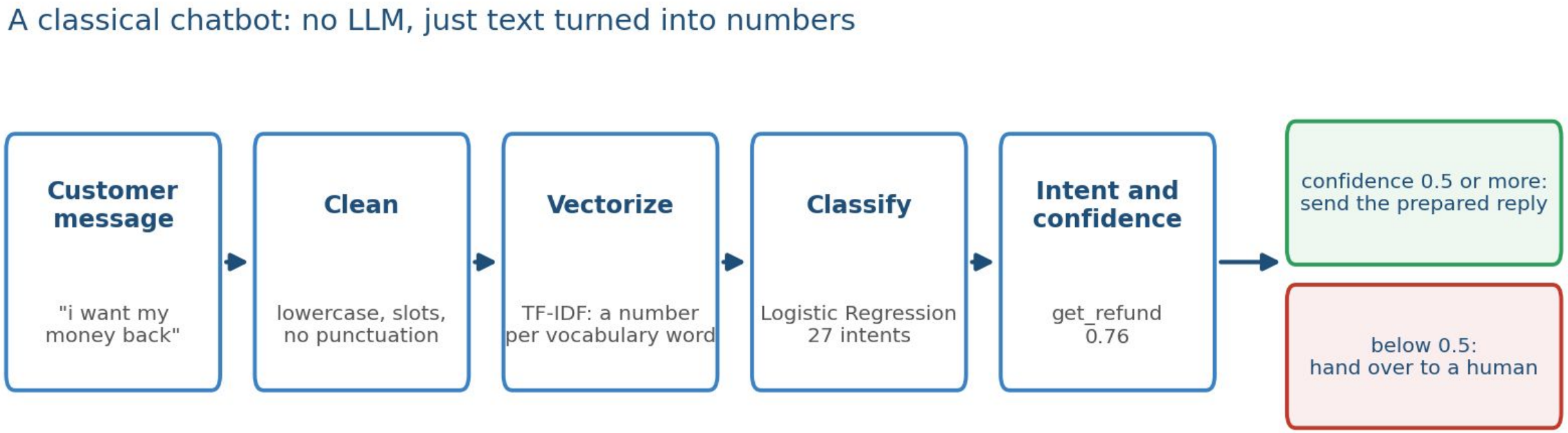
Hour 3

From Classifier to Chatbot

PROJECT

The Design of the Bot

Clean the message, turn it into numbers, classify it, then answer only if the model is sure enough.



EXAMPLE

Prepared Replies and reply()

The dataset already contains example replies. We take the shortest one per intent and turn {{slots}} into [Slots]. reply() answers only above the threshold.

```
def short_reply(intent):
    answers = df.loc[df['intent'] == intent, 'response']
    text = answers.loc[answers.str.len().idxmin()]                # the shortest example reply
    return re.sub(r'\{\{(.*?)\}\}', r'[\1]', text).replace('\n', ' ')   # {{Order Number}} ->
[Order Number]

REPLIES = {intent: short_reply(intent) for intent in model.classes_}
FALLBACK = "Sorry, I am not sure about that. I will pass you to a human agent."

def reply(message, threshold=0.5):
    probs = model.predict_proba(vectorizer.transform([clean(message)]))[0]
    best = probs.argmax()
    if probs[best] < threshold:
        return FALLBACK
    return REPLIES[model.classes_[best]]

print(reply('i want my money back'))
# I can tell that you're unsure about the process of receiving a refund for your money. Don't
# worry, I'm here to assist you every step of the way. To begin, may I ask if you made ...
```

EXAMPLE

The Chat Loop

A few lines turn the function into a conversation. This works in a terminal or a Kaggle cell (type quit to stop).

```
print('Support bot: Hello! How can I help? (type quit to stop)')
while True:
    message = input('You: ')
    if message.strip().lower() == 'quit':
        break
    print('Support bot:', reply(message))
```

RESULTS

A Test Conversation

Real output from reply() with threshold 0.5.

Customer message	Top intent	Confidence	What the bot does
i want my money back	get_refund	0.76	Sends the refund reply
cancel my order please	cancel_order	0.98	Sends the cancel reply
bhai refund do	get_refund	0.57	Sends the refund reply
where is my parcel	delivery_period	0.37	Hands over to a human
what is the weather in lahore	check_refund_policy	0.39	Hands over to a human
hello	get_refund	0.07	Hands over to a human

CONCEPT

Where the Classical Bot Fails

- "where is my parcel" is a normal delivery question, yet scores only 0.37 and goes to a human. The word "parcel" is rare in the training data (about 100 of 26,872 messages), so the model has little to go on
- Two requests in one message get 0.31: the bot can answer at most one intent at a time
- The threshold is a trade-off. At 0.3 the weather question would get a refund-policy reply; at 0.7 even "bhai refund do" is handed to a human
- It cannot handle anything outside its 27 intents, and the replies are fixed text. A language model answers anything, but with other risks. The bridge session covers that

CONCEPT

Three Ways a Support Bot Can Answer

Today's bot is the first row. Where we go next depends on what the business needs.

Approach	How it works	Strength	Weakness
Classifier and prepared replies (today)	Classify the intent, send a prepared reply	Cheap, fast, predictable	Only handles what it was trained on
LLM through an API	Send the message and instructions to a model	Flexible, handles any wording	Costs per token, can hallucinate
LLM with your documents	Look up your policies first, then answer	Grounded in your data	More parts to build and test

PRACTICE

Practice

Chatbot project. Extend your bot in three steps and submit the notebook. (1) Add an "other" intent: write at least 20 off-topic messages (weather, jokes, greetings), add them to the data and retrain. (2) Use your regex from Day 24 to pull an order number out of the message and put it in the reply. (3) Show 10 test messages with the intent, the confidence and the reply. Report what still goes wrong.

PROJECT

How the Project Is Marked

Share this with students at the start of Hour 3.

Part	What a good answer shows
Cleaning	A clean() function with regex, tested on messy messages
Model	TF-IDF plus a classifier, scored on a held-out test set, with the copy check
Confidence	A threshold chosen from real examples, with a fallback to a human
Extension	The other intent and the order-number extractor, both tested
Honesty	A list of messages the bot gets wrong, and why

SUMMARY

Key Takeaways

- Text becomes numbers in steps: clean it, split it into tokens, then count words (bag of words) or weigh them by rarity (TF-IDF)
- An intent classifier is an ordinary classifier on TF-IDF numbers. Ours scored 99.4% on a templated dataset, which says more about the data than about real customers
- Confidence turns a classifier into a safer bot: answer when sure, hand over when not
- Regex and entity extraction add facts the classifier cannot get, such as the order number
- Next: language models, embeddings and attention, then LLM APIs. The bridge deck explains why counting words runs out